Microservice Reference Architecture

Changing the IT Landscape

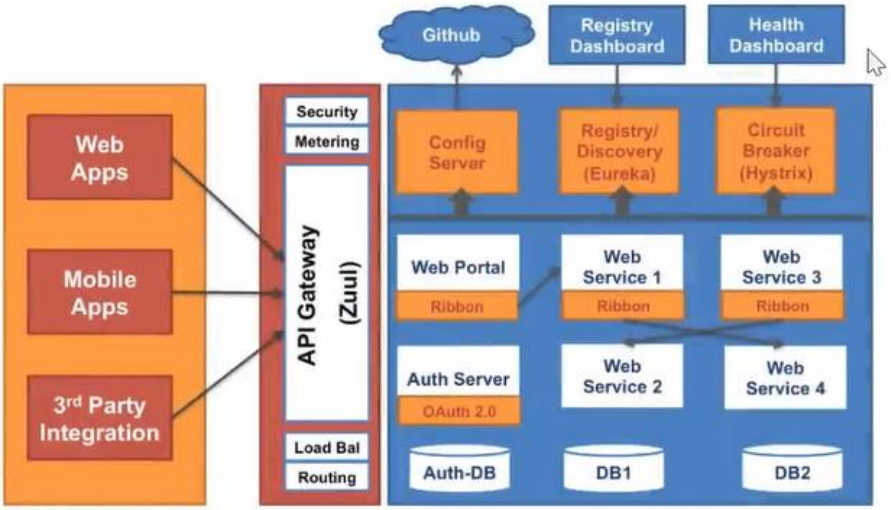
Cloud computing has changed the way infrastructure is built.

The agile movement has brought the principles and practices of CI/CD to various organizations, making **DevOps** a norm

Cost and effort to build software infrastructure has come **down** significantly

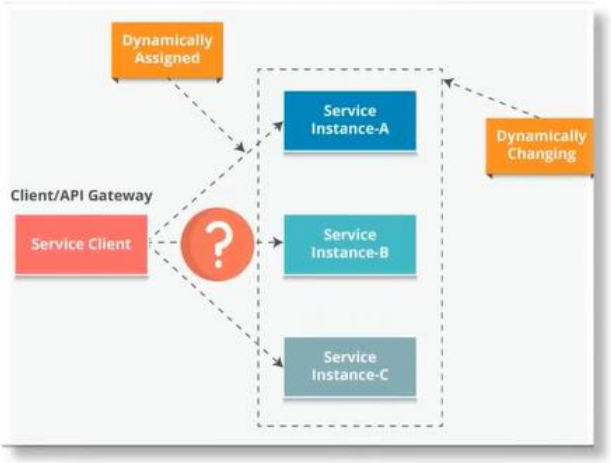
Cultural changes continue to demand **greater agility** from IT organizations

Microservice Reference Architecture

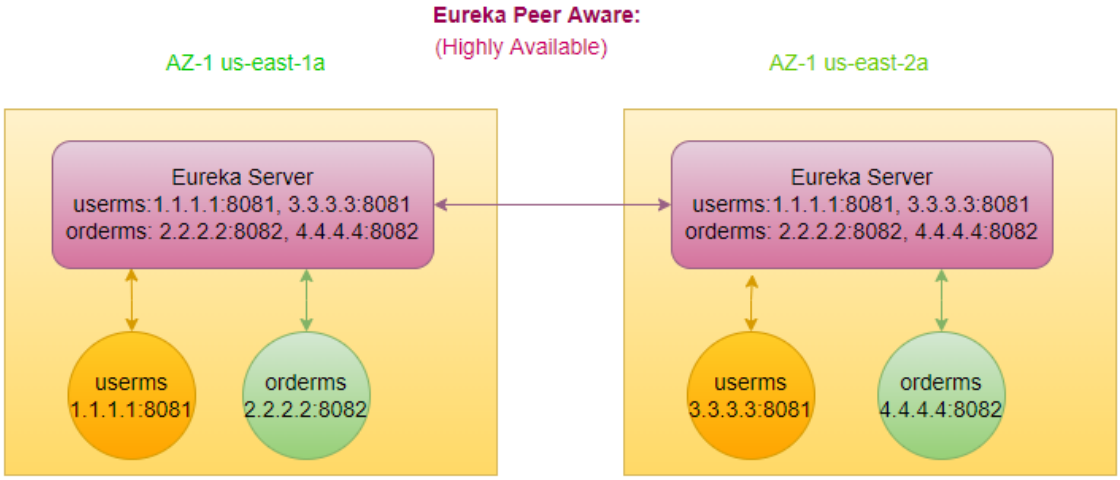


Service Discovery

- **Highly granular** MSA environments
  - Typically involve dozens of services
  - Each service deployed as multiple instances
  - A functionality can involve as many as 10 to 20 calls and be up to 4 or 5 levels deep.
- To solve this mesh, **service discovery** is helpful:
  - Service discovery is critical
  - It enables the reuse of the services as well



Eureka Peer Aware (Highly Available)



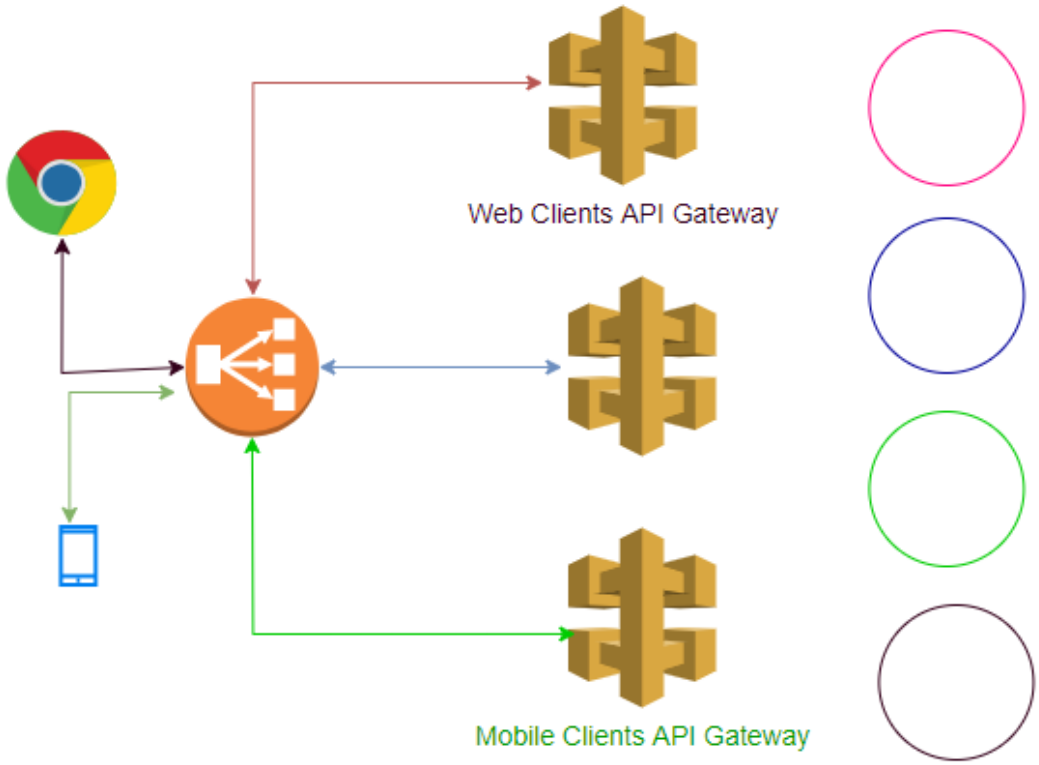
- By default, microservice talk to the eureka server in same AZ.
- If anyone of the Eureka server goes down, initially read the information from the cache, if not available in the cache, then will go to the peer-eureka server (fallback to 2b).
- peer1 pointing to peer2, peer2 point to peer1, that's why they know to each other and can talk to each other, Eureka nothing stores in DB, everything is In-memory data.
- Pass the list of eureka-servers to the client.

```yaml
eureka:
  client:
    serviceUrl:
      defaultZone: http://peer1/eureka/,http://peer2/eureka/,http://peer3/eureka/

---
spring:
  profiles: peer1
eureka:
  instance:
    hostname: peer1

---
spring:
  profiles: peer2
eureka:
  instance:
    hostname: peer2

---
spring:
  profiles: peer3
eureka:
  instance:
    hostname: peer3
```

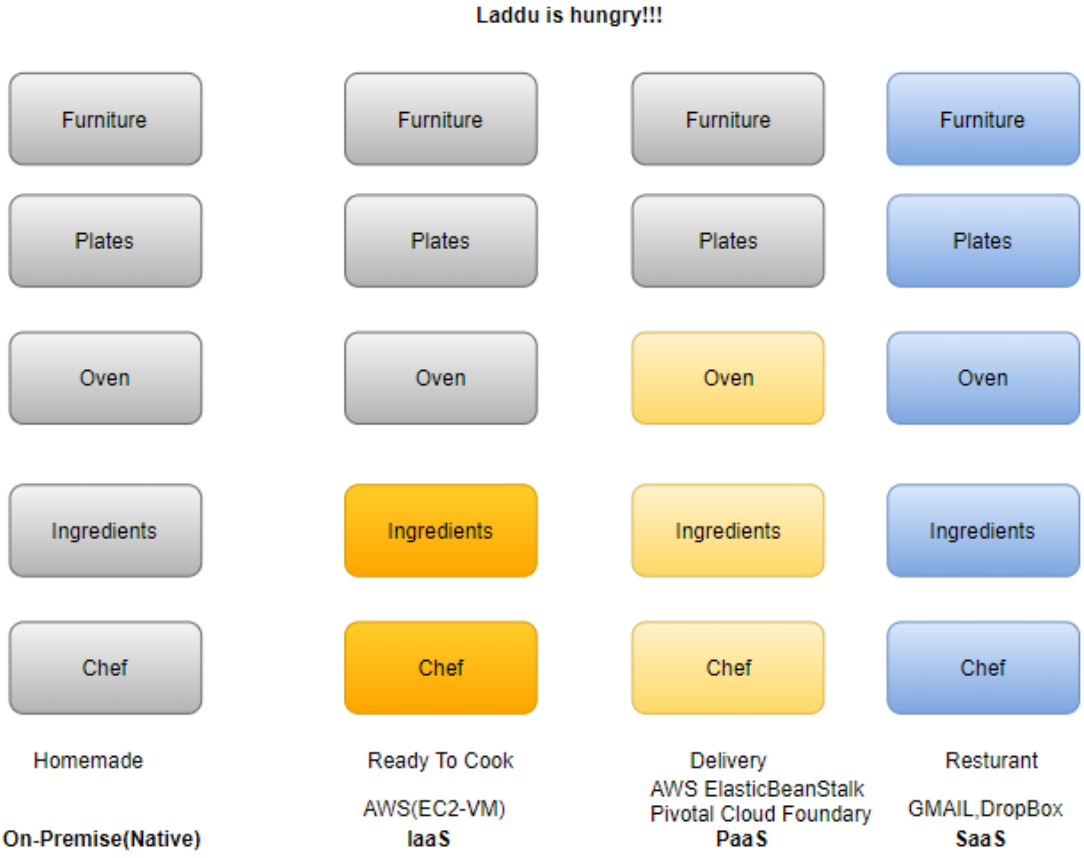

```yaml
eureka:
  client:
    serviceUrl:
      defaultZone: http://peer1/eureka/,http://peer2/eureka/,http://peer3/eureka/

---
spring:
  profiles: peer1
eureka:
  instance:
    hostname: peer1

---
spring:
  profiles: peer2
eureka:
  instance:
    hostname: peer2

---
spring:
  profiles: peer3
eureka:
  instance:
    hostname: peer3
```

How to provide HA to API Gateway?

API Gateway is by default Highly available (serverless) In AWS.



Cloud Service Types



IaaS-> Infra as a Service

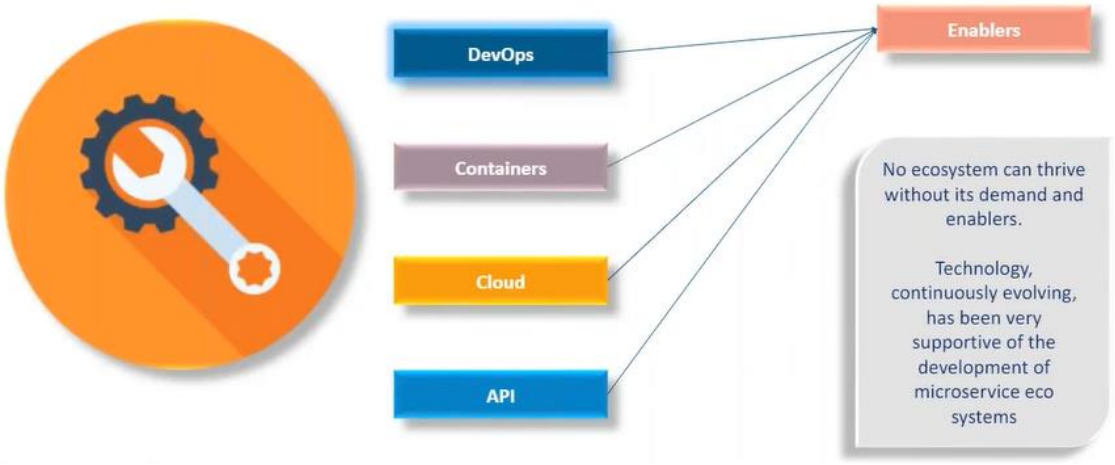
PaaS-> Platform as a Service

SaaS->Software as a Service

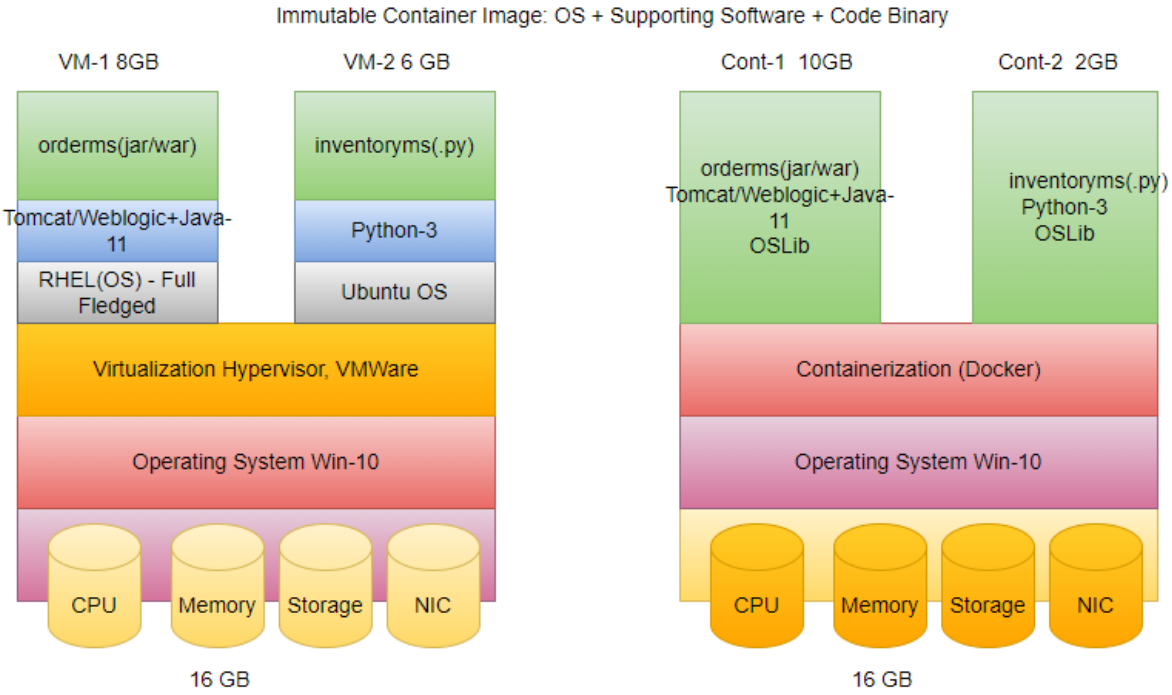
CaaS-> Container as a Service (ECS, EKS)

FaaS-> Function as a Service (Lambda serverless)

Key Microservice Enabler



Cloud (vm Vs. containerization)



In spring world, we create a uber jar (contains all the dependencies we need to run the application)

- Every machine needs four different resources to run properly
  - CPU
  - Memory
  - Storage

    - NIC (Network Interface Card)
- All resources are accessed through the layer of abstraction O/S (kernel).
- On top of that install Hypervisor.
- Create VM on top of Hypervisor, each VM has its own O/S.
- Once you create VM can't do the changes of assigned resources.
- UC: orderms getting 10,000 req/sec (95% utilized), while inventoryms getting 1000 req/sec (40% utilized), we can't use unused resources of userms in inventoryms, use their own resource not the resources of underlying O/S.
- In case of container, uses the underlying O/S resource, dynamic in nature.

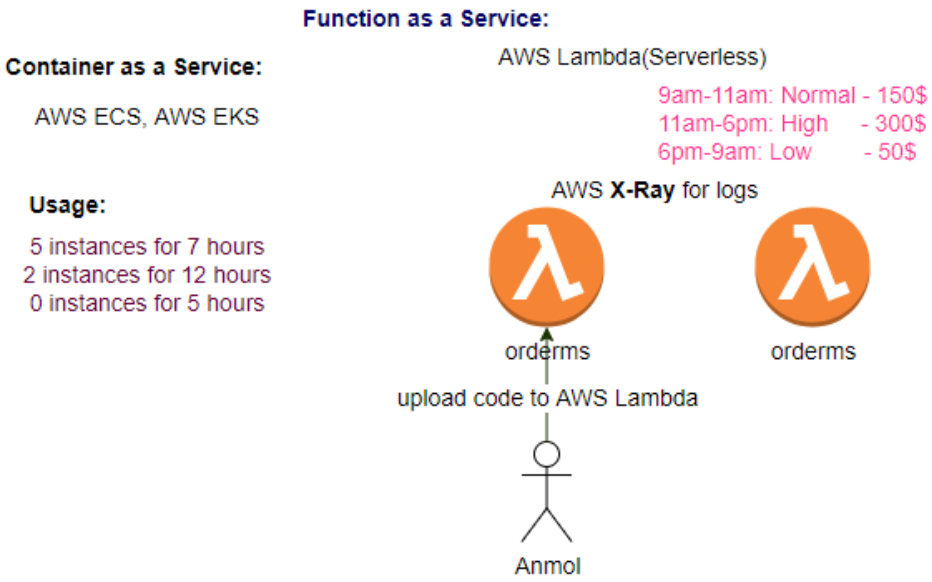
CaaS-> Container as a Service

Managing the containers, using the CaaS, e.g., ECS /EKS

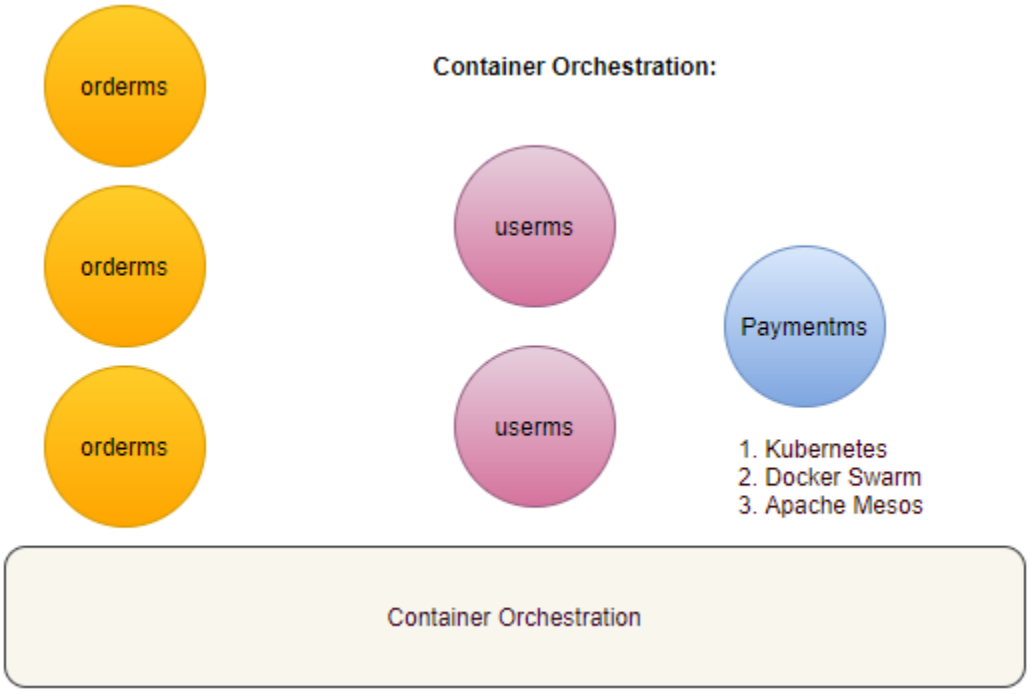
FaaS-> Function as a Service (serverless)

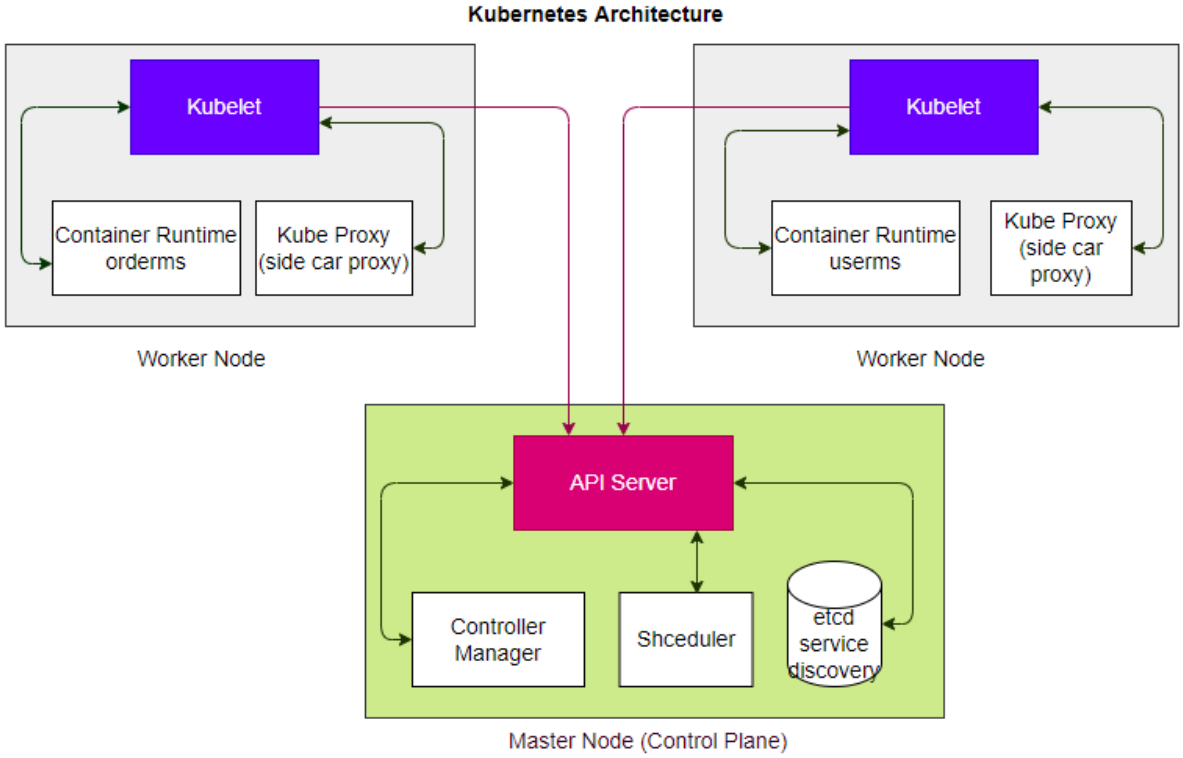
In cloud world, when we say server generally it means VM, is it means serverless there is no VM?

VM managed by the cloud provider. Easier to use, but losing the control.



Container Orchestration:





kube-proxy: traffic forwarding across the backends

kubelet: master agent at worker node

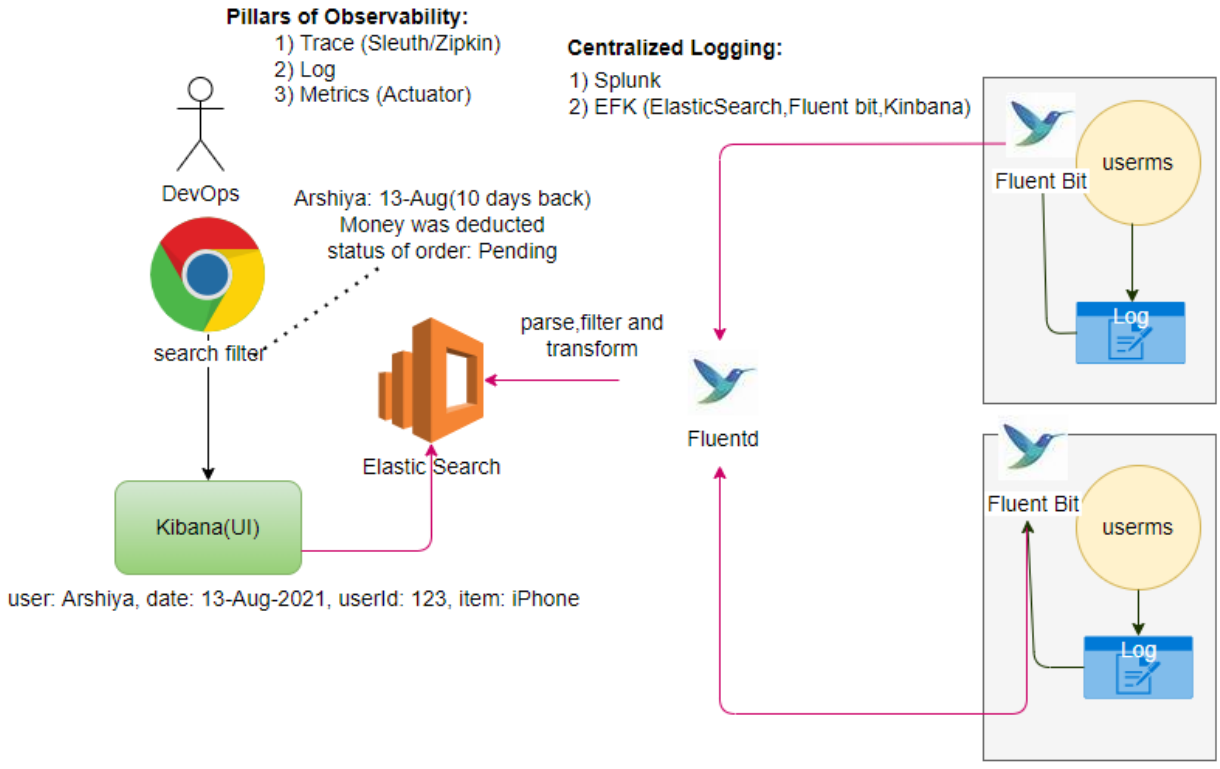
Deployment Strategies

- service per machine (Hardware/VM)
- service per container
- multiple containers per machine

Pillars of Observability

- Trace
- Log
- Metrics

Centralized Logging

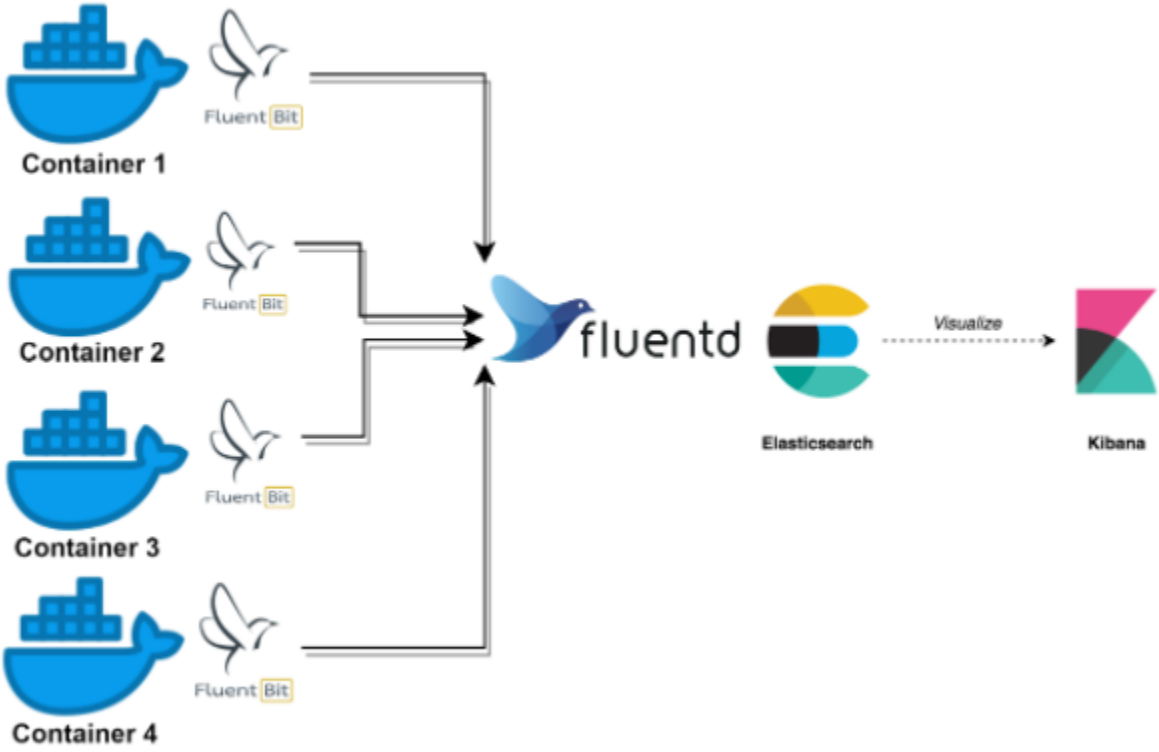


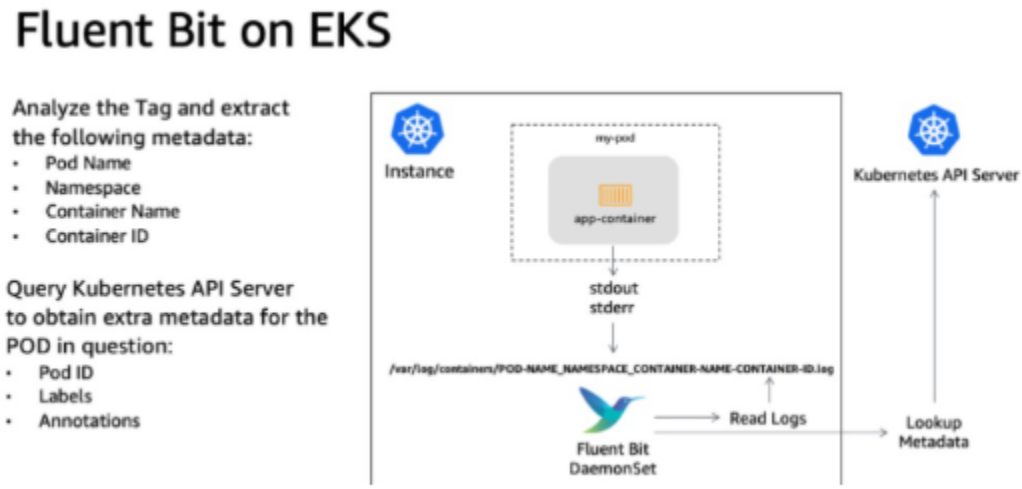
Every ms writes a log, and by default ms writes logs on the node where it is deployed. If we have 100ms and each ms has 5 instances log collection is tedious task.

Solution: We want to put all logs in one central location.

Use the EFK

- Put the logs in Logstash (Repository of Logs)
- Keep the Logs (Unstructured) in Elasticsearch for fast retrieval (Indexing)
- Kibana is for visualize the logs (UI).





UC: search of 10 days logs from 500 instances is not easy task.

How to collect the logs?

Logstash pull the logs from nodes, but nodes are in dynamic in nature

Install the agent (Filebeat/fluent bit ->light weight shipper), runs on each node, keep pushing the logs to the fluentd. Filtering logic must be inside fluentd.

UC: Neeraj access the Kibana and enter the parameters, Kibana sends the request to Elasticsearch.

How to trace the Issue?

Using the Sleuth/Zipkin for tracing

microservice-name, requestId, spanId, isZipkin

payment-ms, reqId-123, spanId-456, true

logging and tracing work together

metricsserver-> prometheus->grafana (nice UI and Graph)

ELK (Centralized logging): NFR

- Elastic Search
- Fluent bit
- Kibana
- Beats (Filebeat): log shipper (agent)

step1: install agent (filebeat) in each instance,

step2: filebeat sends the data to Logstash (aggregate the logs and format)

step3: Logstash forward the logs to Elasticsearch (has its own database).

step4: Kibana (UI)

Distributed Tracking

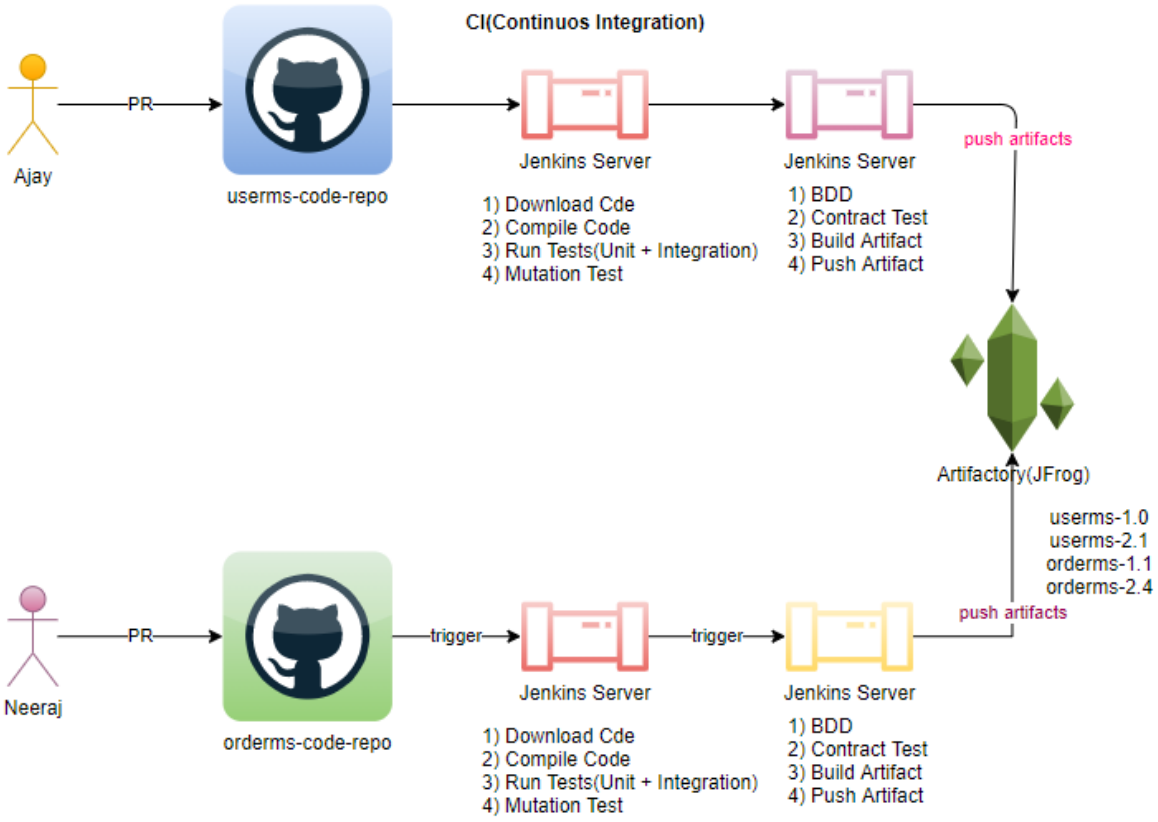
correlationId, traceId

Splunk (Agent on each node to collect the data)

Monitoring

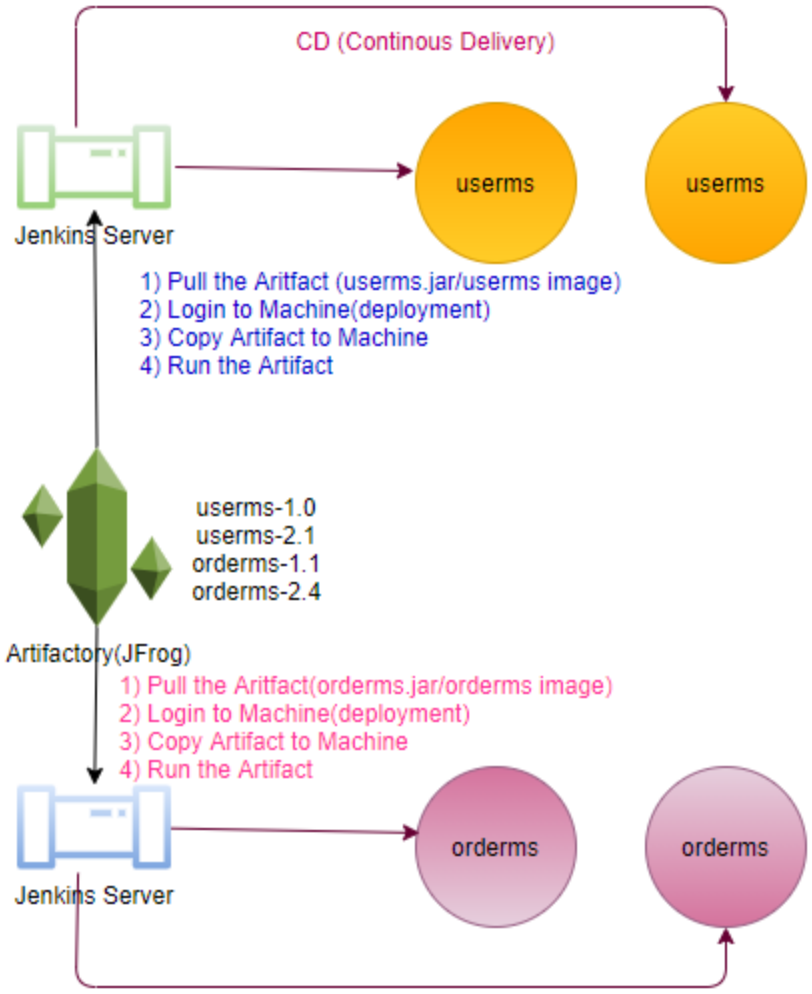
- CloudWatch
- CloudTrail
- X-Ray
- Actuator
- Prometheus
- ELK

CI/CD



Every ms have their own code repository. **Every ms have their own CI/CD pipeline.**

IaaC: Infra as a code



Each microservice Independent to each other, have independent data.

Chaos Engineering In production environment.

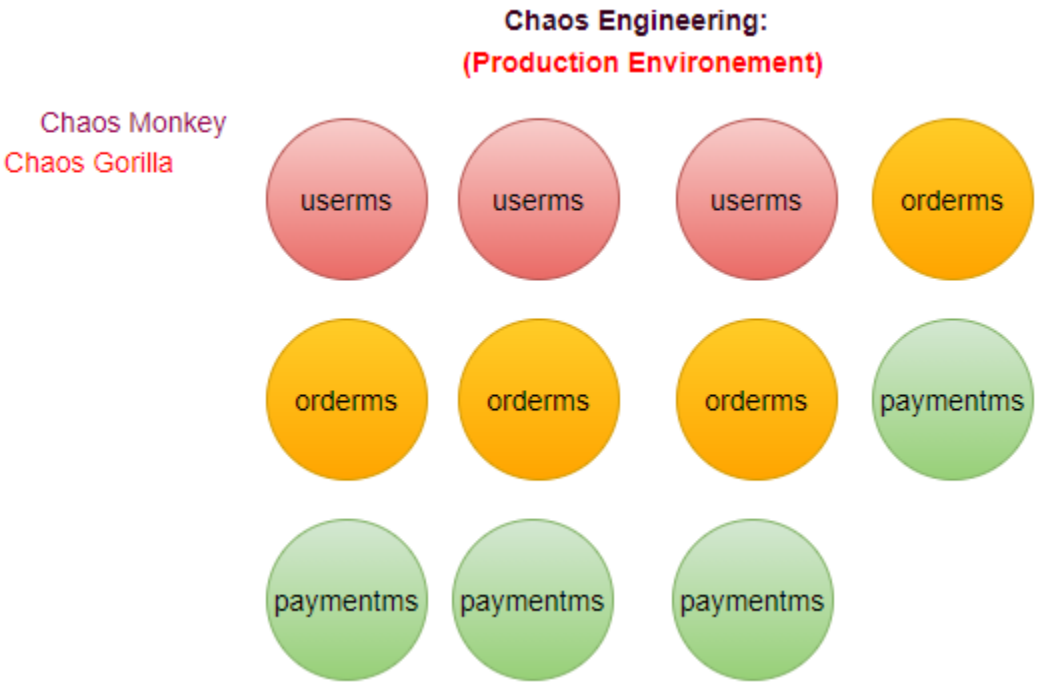
You have several ms running in production environment

Chaos Monkey: Is an agent when it runs start bring down microservices randomly in production, create Chaos in Production, monitoring tool monitor what impact will happen and time will take to back on normal.

Chaos Gorilla: Is an agent when it runs start bring down the whole data center, create Chaos in Production, monitoring tool monitor what impact will happen and time will take to back on normal.

Latency Monkey: Induces artificial delays in our RESTful client-server communication layer to simulates service degradation and measures if upstream service responds appropriately.

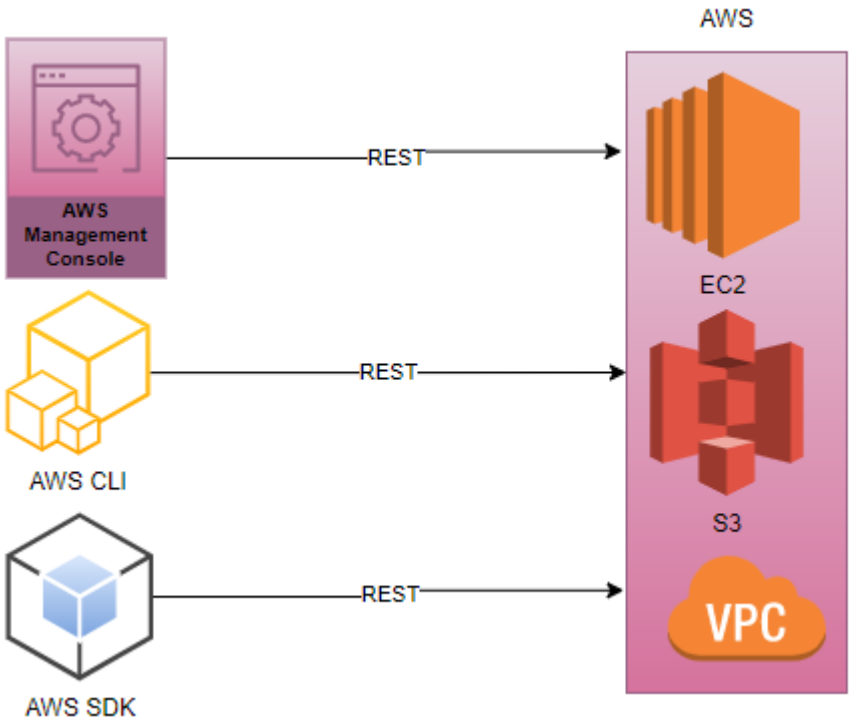
Rather than go things worst in production later, taking preventive actions.

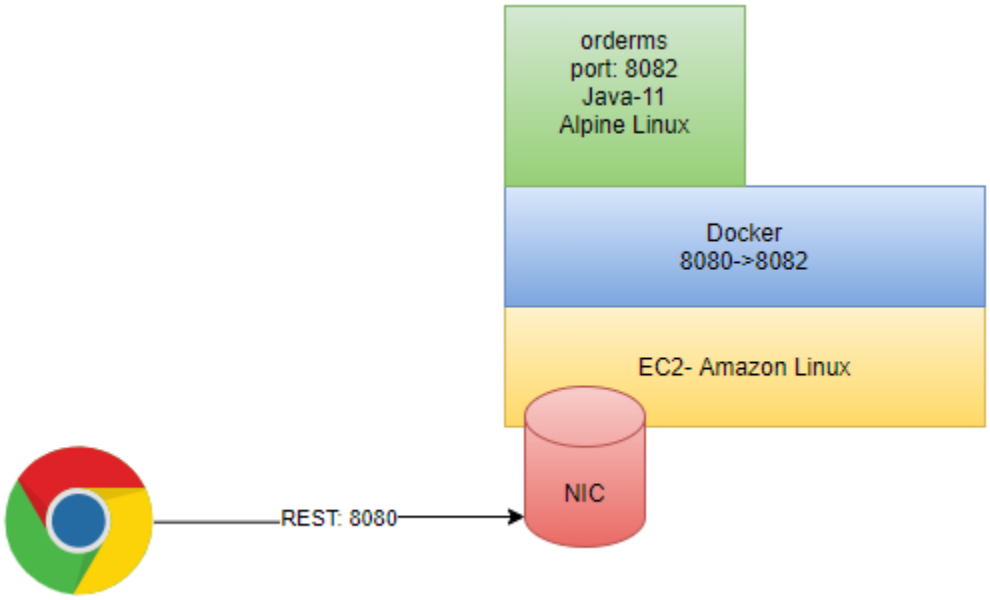


Penetration Testing:

Performance Testing:

AWS (All the cloud services exposed as a Rest Services)





```
Dockerfile:
FROM openjdk:11-jdk-slim
VOLUME /tmp
COPY orderms-0.0.1-SNAPSHOT.jar app.jar
ENTRYPOINT ["java","-Djava.security.egd=file:/dev/./urandom","-jar","/app.jar"]

sudo docker images

sudo docker image build --tag orderms:1.0 .
sudo docker images
sudo docker run -p 8080:8081 -t <docker-image>
sudo docker run -p 8080:8082 -t 3e739e0937c5
sudo docker run -p 8081:8082 -t 3e739e0937c5
```

Access application through the port mapping.

Actuator: (Metrics provided by Actuator)

Dependency: actuator in any ms.

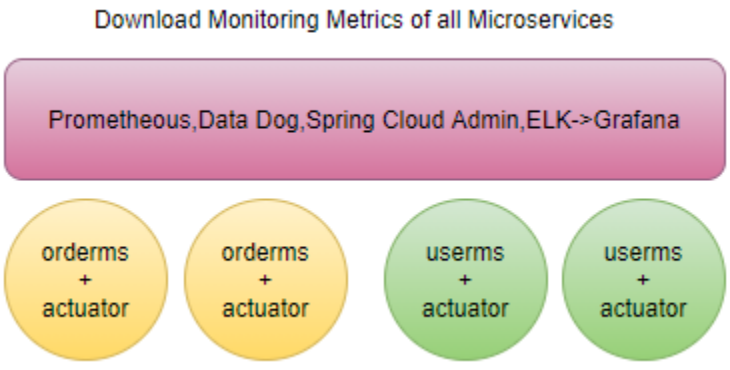
- Just add dependency of actuator is enough to collect the metrics of any microservice.
- expose the actuator dependencies in application. properties.

```
#Actuator
management.endpoints.web.exposure.include=*
```

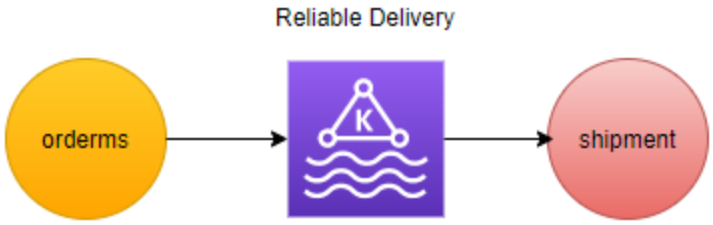
/actuator

/actuator/metrics

This data may be further available to the monitoring tool, which can further display it on some kind of dashboard (prometheus/grafana).



- Service Mesh- Istio:
- Docker/Kubernetes
- Cloud
- Streaming: Kafka Stream, Spark, Apache Flink



Exception Handling?

Learning is continuous journey not the destination.